Lecture Week 07 6COSC020W Applied AI

Machine Learning Fundamentals

with the Iris flowers dataset

Features, the ML pipeline, linear regression, gradient descent, decision trees, k-means, reinforcement learning and evaluation metrics.

Dr. Hamed Hamzeh



ROADMAP

What we will cover today

01

Foundations

Definition of ML, features, dependent and independent variables, types of ML

02

Data pipeline

The complete pipeline, pre-processing, splitting, feature engineering, scaling

03

Linear regression

The formula, slope and intercept, loss function, gradient descent, evaluation

04

Decision trees

How a tree is grown and how the Gini index chooses each split

05

Unsupervised & RL

K-means clustering, the elbow method and the RL agent loop

06

Evaluation

Confusion matrix, accuracy, precision, recall and F1-score

PART 1

Foundations



What machine learning is, the language we use to describe data, and the three families of learning.

- Definition
- Features
- Variables
- Types of ML

What is machine learning?

Instead of writing a rule for every situation, we give the computer examples and let an algorithm learn a model from them.

Traditional programming




```
IF petal length < 2.5 cm
THEN species = Setosa

A programmer writes every rule by hand.
```

Machine learning



We supply measurements of flowers whose species is already known.

The algorithm discovers the rules (the model) by itself.

A formal definition: task, experience, performance

> *“A computer program is said to learn from experience E with respect to some task T and performance measure P, if its performance at T, as measured by P, improves with experience E.”*
>
> — Tom Mitchell, Machine Learning (1997)

T



Task

What must the system do?

Predict the species of an iris flower from its four measurements.

E



Experience

What does it learn from?

150 flowers that botanists have already measured and labelled.

P



Performance

How do we score it?

Accuracy — the share of unseen flowers classified correctly.

Meet the Iris flowers dataset

150
flower observations

4
numerical features (cm)

3
species, 50 of each

0
missing values

```python
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame          # 150 rows x 5 columns
X = iris.data            # the 4 features
y = iris.target          # 0, 1, 2  (species)
print(df.head())
```

Sepal length	Sepal width	Petal length	Petal width	Species
5.1	3.5	1.4	0.2	Setosa
4.9	3.0	1.4	0.2	Setosa
6.0	2.9	4.5	1.5	Versicolor
5.7	2.8	4.1	1.3	Versicolor
6.5	3.0	5.8	2.2	Virginica
6.3	3.3	6.0	2.5	Virginica

A classic since 1936

Collected by Edgar Anderson and made famous by the statistician Ronald Fisher. Small, clean and easy to visualise — ideal for learning.

What is a feature?

Feature: a measurable property of each observation that the model uses as input. Also called an attribute, predictor or input variable. Each row is one observation; each column is one feature.

FEATURES — the inputs X				LABEL / TARGET — y
sepal length	sepal width	petal length	petal width	species
5.1	3.5	1.4	0.2	Setosa
6.3	3.3	6.0	2.5	Virginica

Sepal length

4.3 – 7.9 cm

mean 5.84 · std 0.83

Sepal width

2 – 4.4 cm

mean 3.06 · std 0.43

Petal length

1 – 6.9 cm

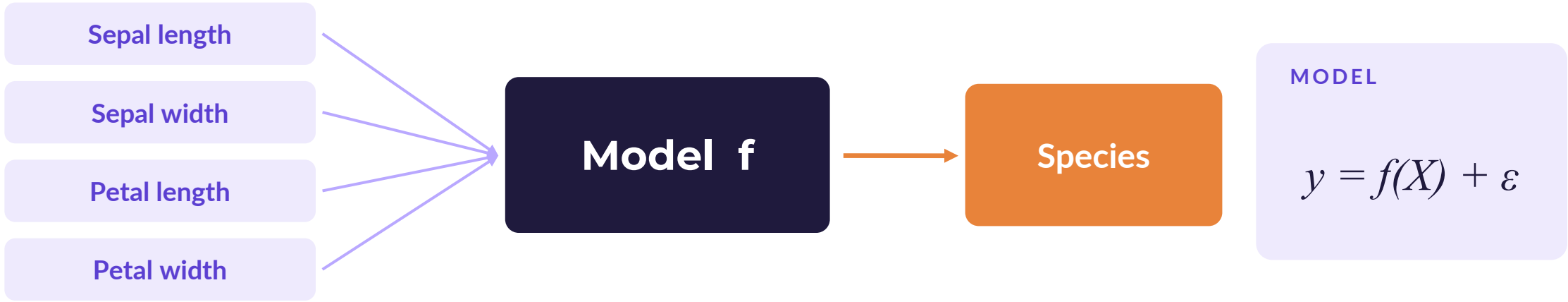
mean 3.76 · std 1.76

Petal width

0.1 – 2.5 cm

mean 1.2 · std 0.76

Independent and dependent variables



Independent variables (X)

- The inputs we measure or control
- Also called features or predictors
- Iris: the four sepal and petal measurements
- Usually a matrix: 150 rows × 4 columns

Dependent variable (y)

- The output we want to predict
- Also called the target, label or response
- Its value depends on X
- Iris: species — or petal width in our regression example

Three main types of machine learning



Supervised

Every training example has a known answer — a label or target value.

EXAMPLES

- Predict Iris species (classification)
- Predict petal width (regression)
- Spam filtering, house prices



Unsupervised

No labels. The algorithm discovers groups or structure on its own.

EXAMPLES

- Group Iris flowers without being told the species
- Customer segmentation



Reinforcement

An agent acts in an environment and learns from rewards and penalties.

EXAMPLES

- Game-playing agents
- Robot navigation
- Traffic-light control

The three types side by side

	Supervised	Unsupervised	Reinforcement
Training data	Features + labels (X, y)	Features only (X)	States, actions, rewards
Goal	Predict y for new inputs	Find hidden structure	Learn a policy that maximises reward
Feedback	Correct answer for every example	None	Delayed reward signal
Typical tasks	Classification, regression	Clustering, dimensionality reduction	Control, games, robotics
Algorithms today	Linear regression, decision tree	K-means	Q-learning (concept)
With Iris	Predict species from measurements	Group flowers ignoring species	Not a natural fit — no actions

Rule of thumb: have labels → supervised. No labels → unsupervised. Learning by trial and error → reinforcement.

PART 2

Pipeline & data preparation

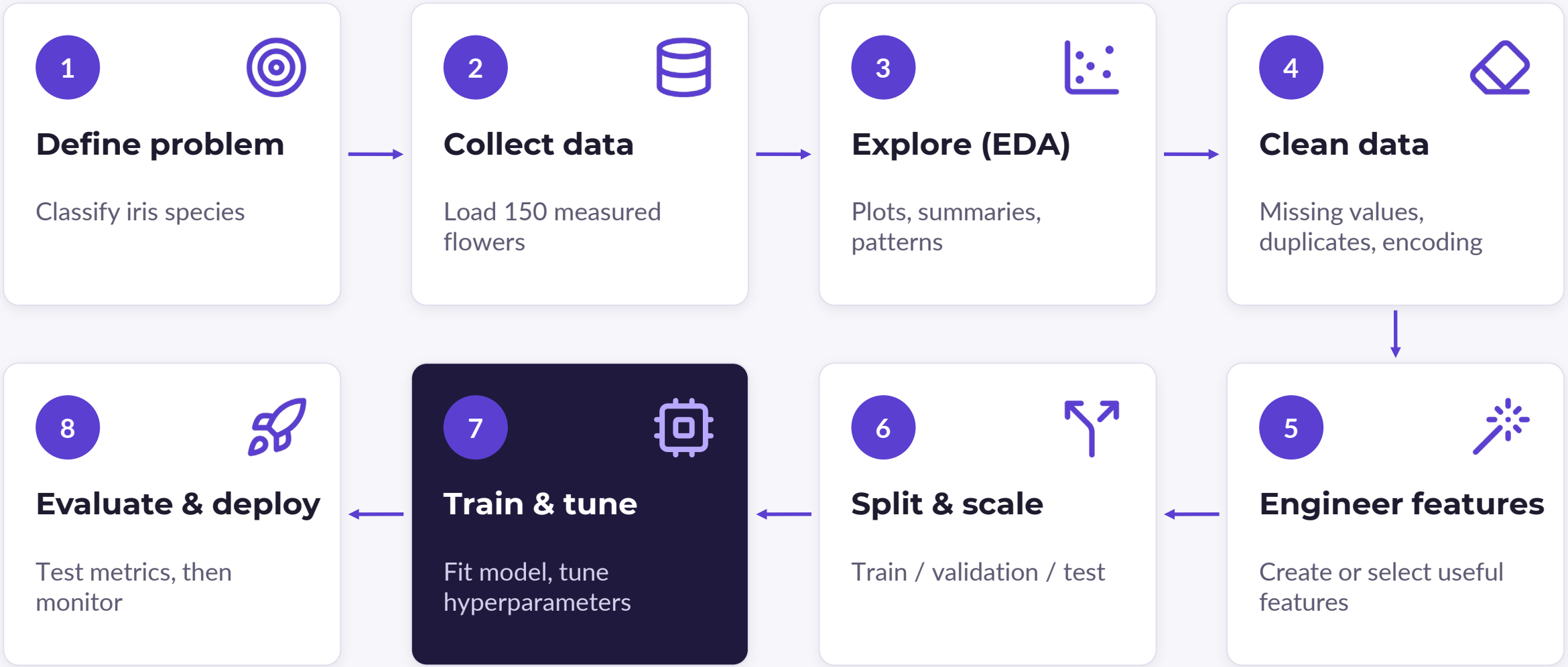


Most of the work in a real project happens before a model is ever trained.

- Pipeline
- Pre-processing
- Splitting
- Feature engineering
- Scaling

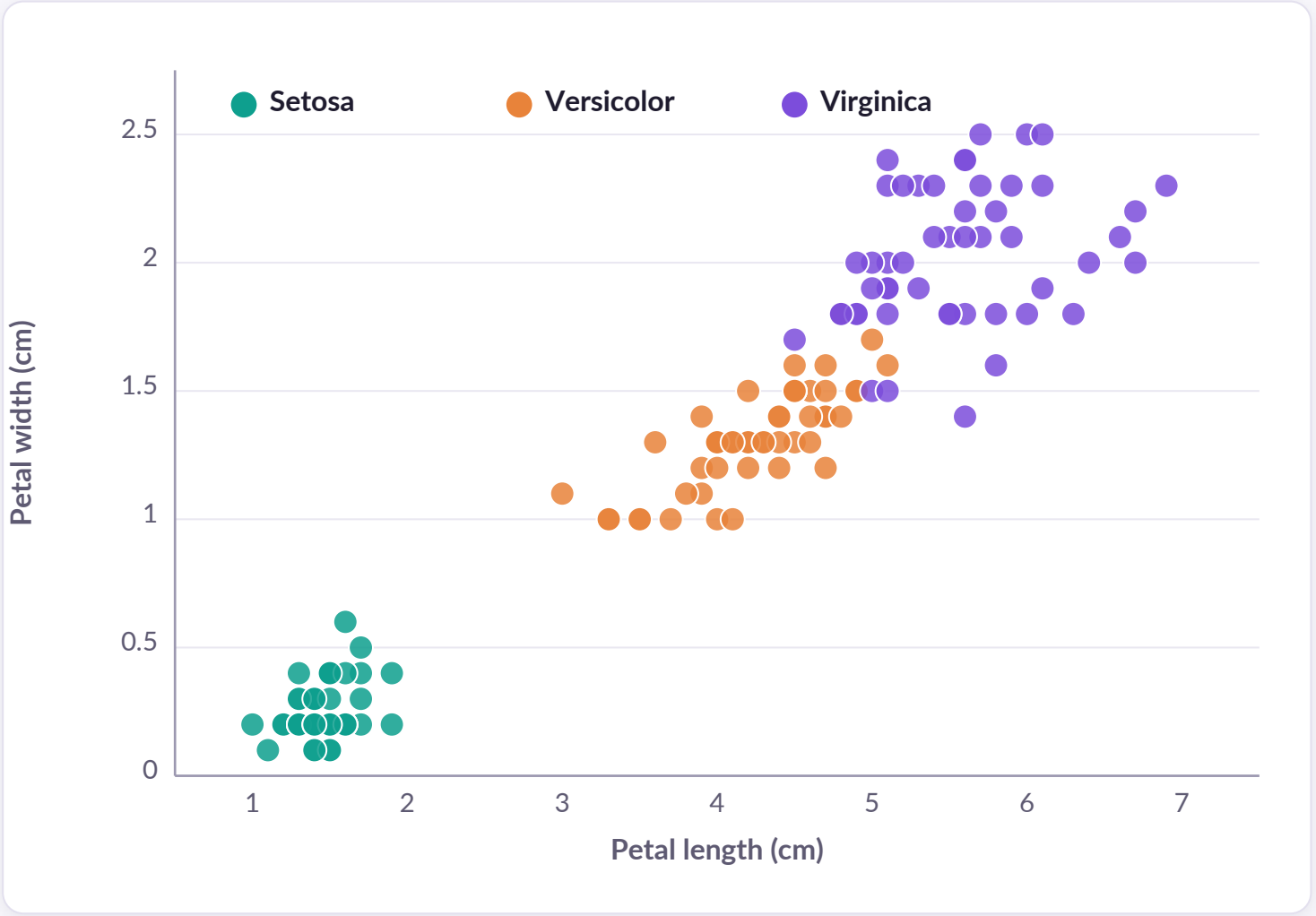
PIPELINE

The complete machine learning pipeline



EXPLORE THE DATA

Exploratory analysis: petals separate the species



Setosa stands apart

All 50 Setosa flowers have petal length below 2 cm.

Two species overlap

Versicolor and Virginica share a boundary region where models make mistakes.

Why explore first?

EDA reveals patterns, outliers and which features matter before modelling.

Pre-processing: clean the data before modelling

Missing values

Drop rows or impute with the mean, median or mode.

0 missing in Iris

Duplicates

Identical rows can bias training and leak into the test set.

1 duplicate row

Outliers

Detect with the IQR rule or z-scores; investigate before removing.

4 in sepal width (IQR)

Encoding

Models need numbers: map text categories to integers or one-hot columns.

setosa → 0, versicolor → 1 …

```python
df = iris.frame

# 1. missing values and duplicates
df.isnull().sum()       # 0 per column
df.duplicated().sum()   # 1
df = df.drop_duplicates()

# 2. outliers with the IQR rule
sw = df["sepal width (cm)"]
q1, q3 = sw.quantile([0.25, 0.75])
iqr = q3 - q1
low, high = q1 - 1.5*iqr, q3 + 1.5*iqr
outliers = df[(sw < low) | (sw > high)]

# 3. encode text labels
from sklearn.preprocessing import LabelEncoder
y = LabelEncoder().fit_transform(names)
```

Train, validation and test sets

Stratified 60 / 20 / 20 split of 150 flowers



Training set

The model learns its parameters (slope, splits, centroids) from these examples.

Validation set

Used to compare models and tune hyperparameters such as tree depth.

Test set

Locked away until the end — gives an honest estimate on unseen data.

python

```python
from sklearn.model_selection import train_test_split

# 1) hold out 20% as the final test set
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)

# 2) split the rest into train (60%) and validation (20%)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, stratify=y_temp,
    random_state=42)

print(len(X_train), len(X_val), len(X_test))   # 90 30 30
```

Feature engineering: create better inputs

Create

Combine features, e.g. petal area = length × width

Transform

Log, square root or polynomial terms

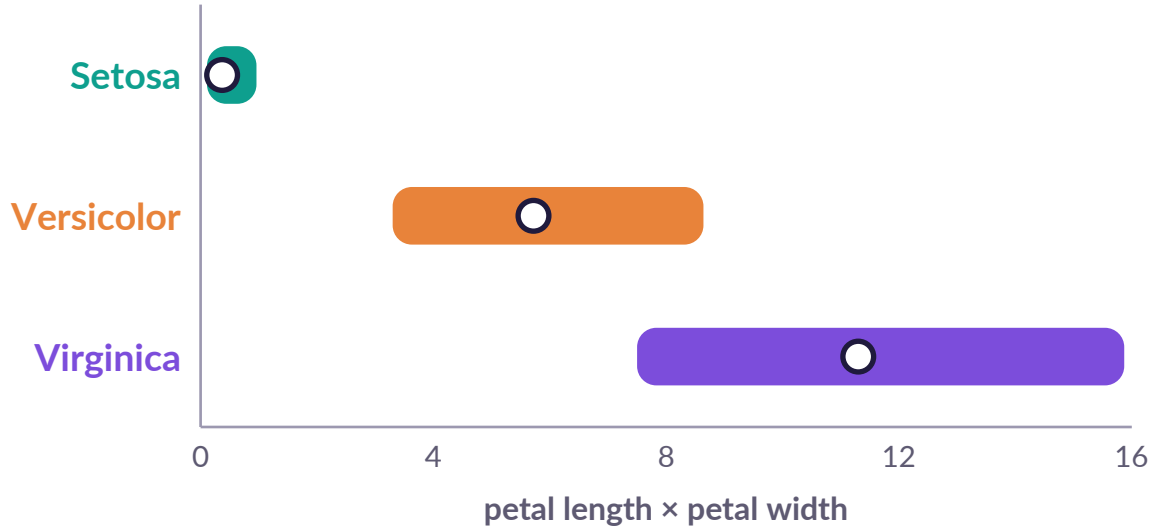
Encode

Turn categories into numbers (one-hot, label)

Select

Keep informative features, drop noisy ones

New feature: petal area (cm²) — range per species




```python
df["petal_area"] = (df["petal length (cm)"]
                    * df["petal width (cm)"])
```

Why scale features?

Feature	Min	Max	Mean	Std
Sepal length	4.3	7.9	5.84	0.83
Sepal width	2.0	4.4	3.06	0.43
Petal length	1.0	6.9	3.76	1.76
Petal width	0.1	2.5	1.20	0.76

Petal length spans 5.9 cm but petal width only 2.4 cm — features live on different scales.

Distances get distorted

Algorithms that measure distance or use gradients are dominated by the feature with the largest range. Scaling puts every feature on an equal footing and helps gradient descent converge faster.

Scaling needed

- KNN
- K-means
- SVM
- Gradient descent
- Neural networks
- PCA

Not needed

- Decision trees
- Random forest
- Gradient-boosted trees
- Naive Bayes

Standardisation vs normalisation

Standardisation

StandardScaler

$$z = (x - \mu) / \sigma$$

- Result has mean 0 and standard deviation 1
- No fixed range — values usually between −3 and 3
- Default choice; less affected by outliers

Min–max normalisation

MinMaxScaler

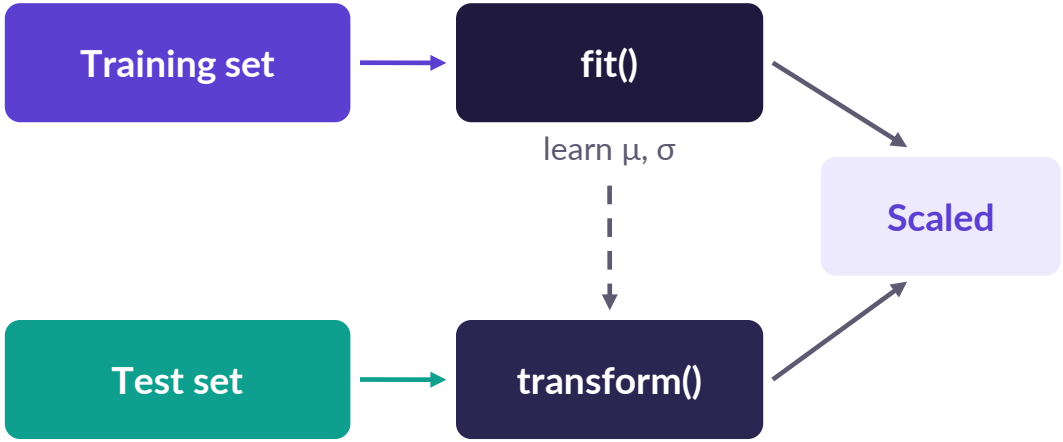
$$x' = (x - min) / (max - min)$$

- Squashes every value into the range [0, 1]
- Useful for neural networks and image pixels
- Sensitive to extreme values

The first Setosa flower before and after scaling

	Sepal length	Sepal width	Petal length	Petal width
Raw (cm)	5.1	3.5	1.4	0.2
Standardised (z)	-0.90	1.02	-1.34	-1.32
Min–max [0, 1]	0.22	0.62	0.07	0.04

Scale correctly: fit on the training set only




```python
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)   # fit + transform
X_test_s  = scaler.transform(X_test)        # transform only

# Better: let a pipeline do it for you
model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5),
)
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

Avoid data leakage

If the scaler is fitted on all 150 flowers, the test set's mean and spread leak into training and the test score becomes too optimistic. Use a Pipeline so this cannot happen by accident.

PART 3

Supervised learning: linear regression

Predicting a continuous number — and the mathematics of how a model learns from its errors.



- Formula
- Slope & intercept
- Loss function
- Gradient descent
- Evaluation

Two kinds of supervised problems

Classification

Target is a category

Spam or legitimate email	`spam / ham`
Disease diagnosis	`yes / no`
Iris species	`3 classes`

Regression

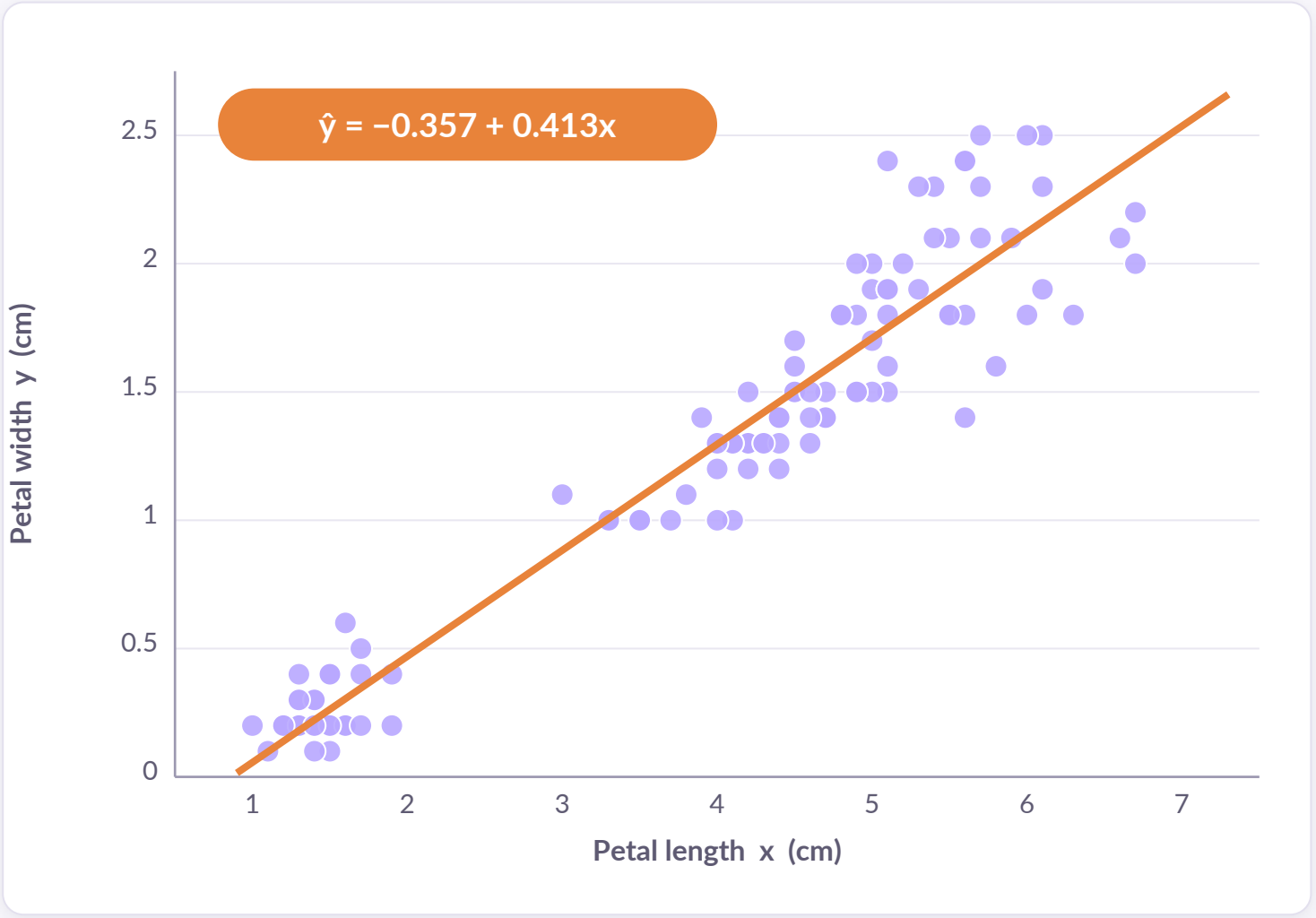
Target is a continuous number

House prices	`£245,000`
Temperature	`17.8 °C`
Petal width	`1.5 cm`

Iris: predict the species (Setosa, Versicolor, Virginica) from four measurements.

Iris: predict petal width (cm) from petal length — our running example.

Fit a straight line through the data



Strong linear trend

Correlation r = 0.963 between petal length and width on 120 training flowers.

One line summarises it

Linear regression finds the single straight line that best fits the points.

“Best” = smallest errors

The line minimises the squared vertical distances to the points.

The linear regression formula

$$\hat{y} = \beta_0 + \beta_1 x$$

- $\hat{y}$ — predicted value
- β_0 — intercept
- β_1 — slope
- x — feature

Many features:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$$

Slope β_1

0.413

For every extra 1 cm of petal length, predicted petal width rises by about 0.41 cm.

Intercept β_0

−0.357

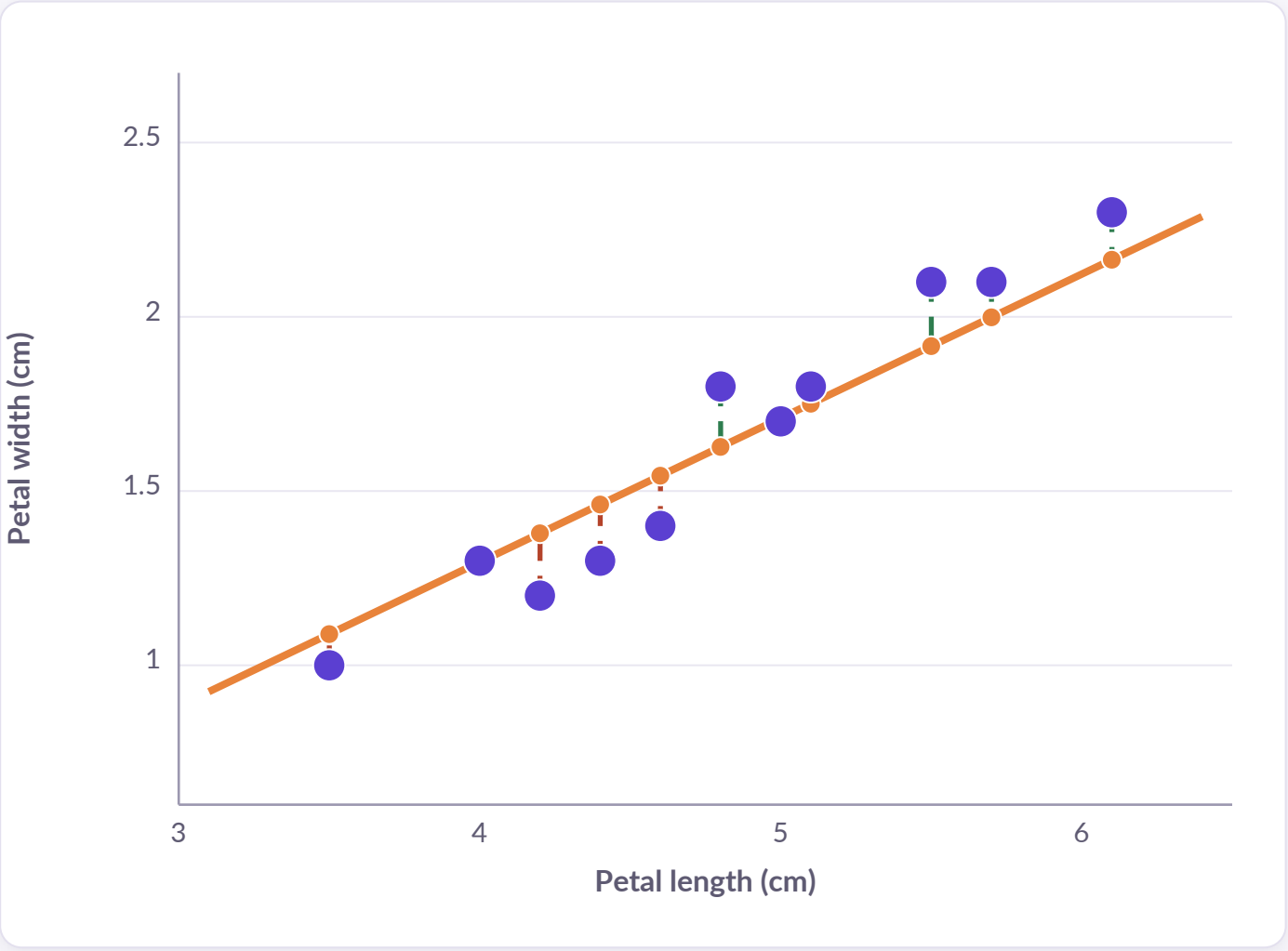
The predicted width when x = 0. Here it is not physically meaningful — it just positions the line.

Prediction

1.50 cm

Petal length 4.5 cm: $\hat{y} = -0.357 + 0.413 \times 4.5$

Residuals: how wrong is the line for each point?



RESIDUAL

$$e_i = y_i - \hat{y}_i$$

Actual value y_i
the real measured petal width

Predicted value $\hat{y}_i$
the point on the line

Positive residual
point above the line — under-prediction

Negative residual
point below the line — over-prediction

The loss function: mean squared error

LOSS FUNCTION $J(\beta_0, \beta_1)$

$$MSE = (1/n)\,\Sigma\,(y_i - \hat{y}_i)^2$$

Squaring makes every error positive

so errors above and below the line do not cancel out.

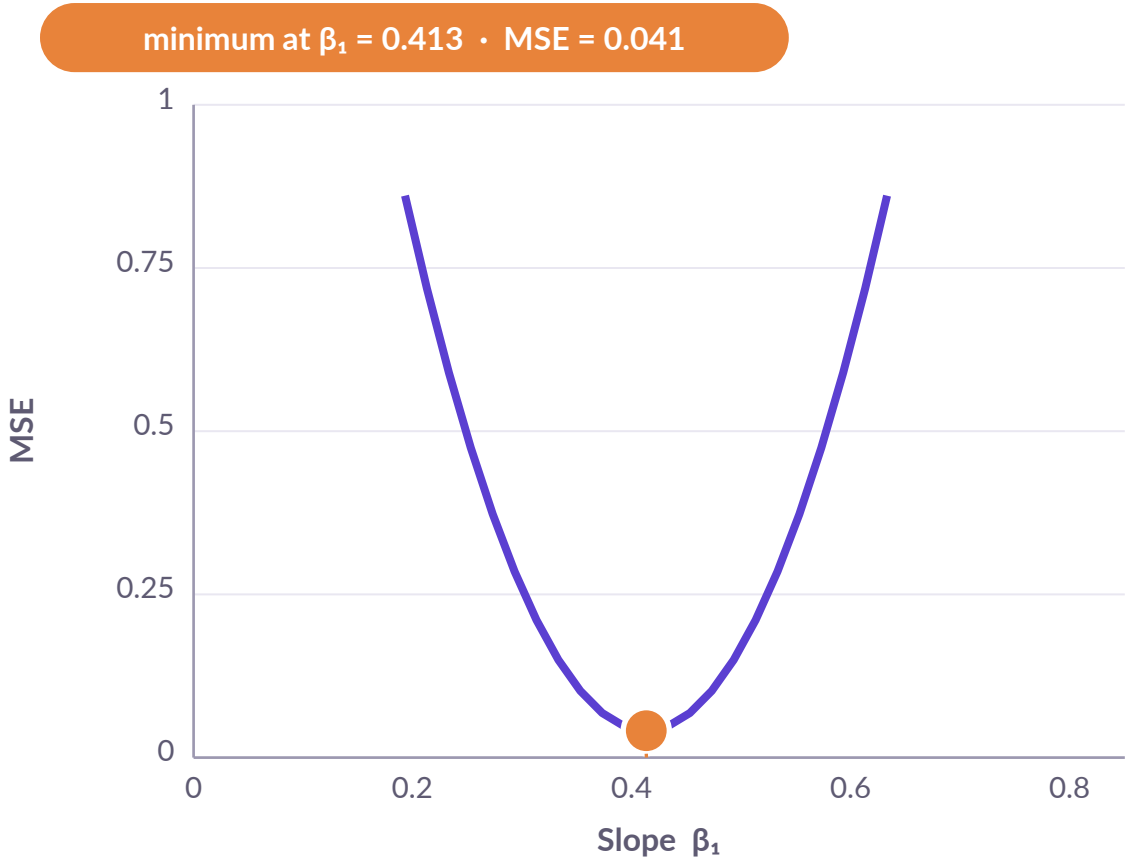
Big errors are punished more

an error of 2 costs 4; an error of 0.5 costs only 0.25.

Smooth bowl shape

it has one minimum, which we can find with calculus or gradient descent.

Training MSE as the slope changes (intercept fixed)



Calculating the best slope and intercept

Setting the derivatives of the MSE to zero gives a closed-form answer, known as ordinary least squares (OLS).

SLOPE

$$\beta_1 = \Sigma\,(x_i - \bar{x})(y_i - \bar{y}) \;/\; \Sigma\,(x_i - \bar{x})^2$$

INTERCEPT

$$\beta_0 = \bar{y} - \beta_1\,\bar{x}$$

In words

The slope is how much x and y vary together (covariance) divided by how much x varies on its own (variance). The line always passes through the point $(\bar{x}, \bar{y})$.

IRIS TRAINING SET · n = 120

1. Means
   $\bar{x} = 3.727 \quad \bar{y} = 1.183$
2. Sum of cross-products
   $\Sigma(x-\bar{x})(y-\bar{y}) = 151.00$
3. Sum of squares of x
   $\Sigma(x-\bar{x})^2 = 365.41$
4. Slope and intercept
   $\beta_1 = 0.4132 \quad \beta_0 = -0.357$

Worked example: five flowers by hand

Flower	x (length)	y (width)	$x - \bar{x}$	$y - \bar{y}$	$(x - \bar{x})(y - \bar{y})$	$(x - \bar{x})^2$
1	1.4	0.2	−2.48	−1.14	2.827	6.150
2	3.5	1.0	−0.38	−0.34	0.129	0.144
3	3.8	1.1	−0.08	−0.24	0.019	0.006
4	5.1	2.0	1.22	0.66	0.805	1.488
5	5.6	2.4	1.72	1.06	1.823	2.958
Σ / mean	**$\bar{x}$ = 3.88**	**$\bar{y}$ = 1.34**			5.604	10.748

SLOPE

$$\beta_1 = 5.604 / 10.748 = 0.521$$

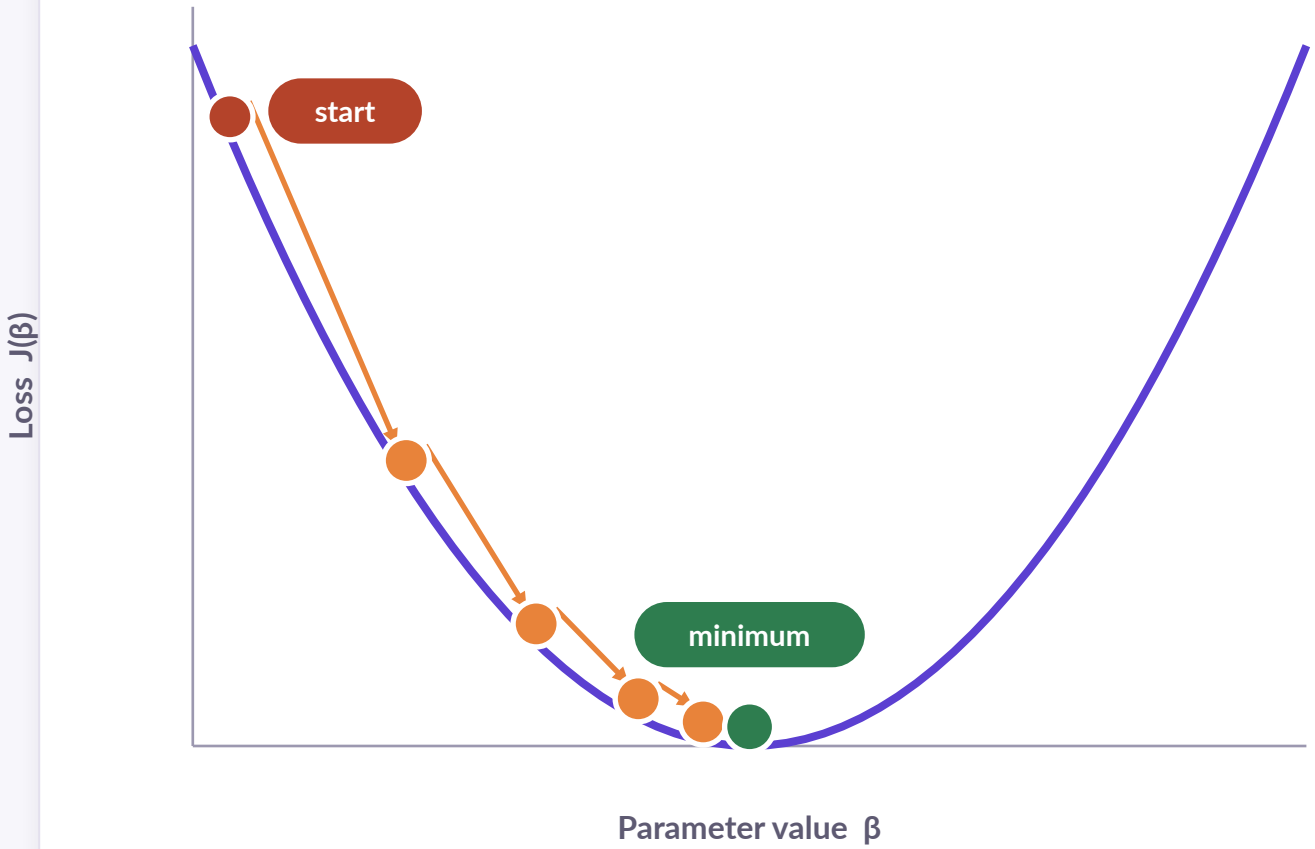
INTERCEPT

$$\beta_0 = 1.34 - 0.521 \times 3.88 = -0.681$$

FITTED LINE

$$\hat{y} = -0.681 + 0.521x$$

Gradient descent: walking downhill on the loss



Each step moves against the slope; steps shrink as the curve flattens.

1. Start anywhere

Initialise β_0 and β_1 (for example, at 0).

2. Measure the slope

Compute the gradient — the direction in which the loss rises fastest.

3. Take a step downhill

Move each parameter a small amount in the opposite direction.

4. Repeat until flat

Stop when the loss stops decreasing (convergence).

The update rules for linear regression

STEP 1 · GRADIENTS OF THE MSE

$$\partial J/\partial \beta_0 = (2/n)\ \Sigma\ (\hat{y}_i - y_i)$$
$$\partial J/\partial \beta_1 = (2/n)\ \Sigma\ (\hat{y}_i - y_i) \cdot x_i$$

STEP 2 · UPDATE BOTH PARAMETERS

$$\beta_0 \leftarrow \beta_0 - \alpha \cdot \partial J/\partial \beta_0$$
$$\beta_1 \leftarrow \beta_1 - \alpha \cdot \partial J/\partial \beta_1$$

α	$\partial J/\partial \beta$	$\leftarrow$
learning rate — the step size	gradient — the slope of the loss	replace with the new value

python

```python
import numpy as np

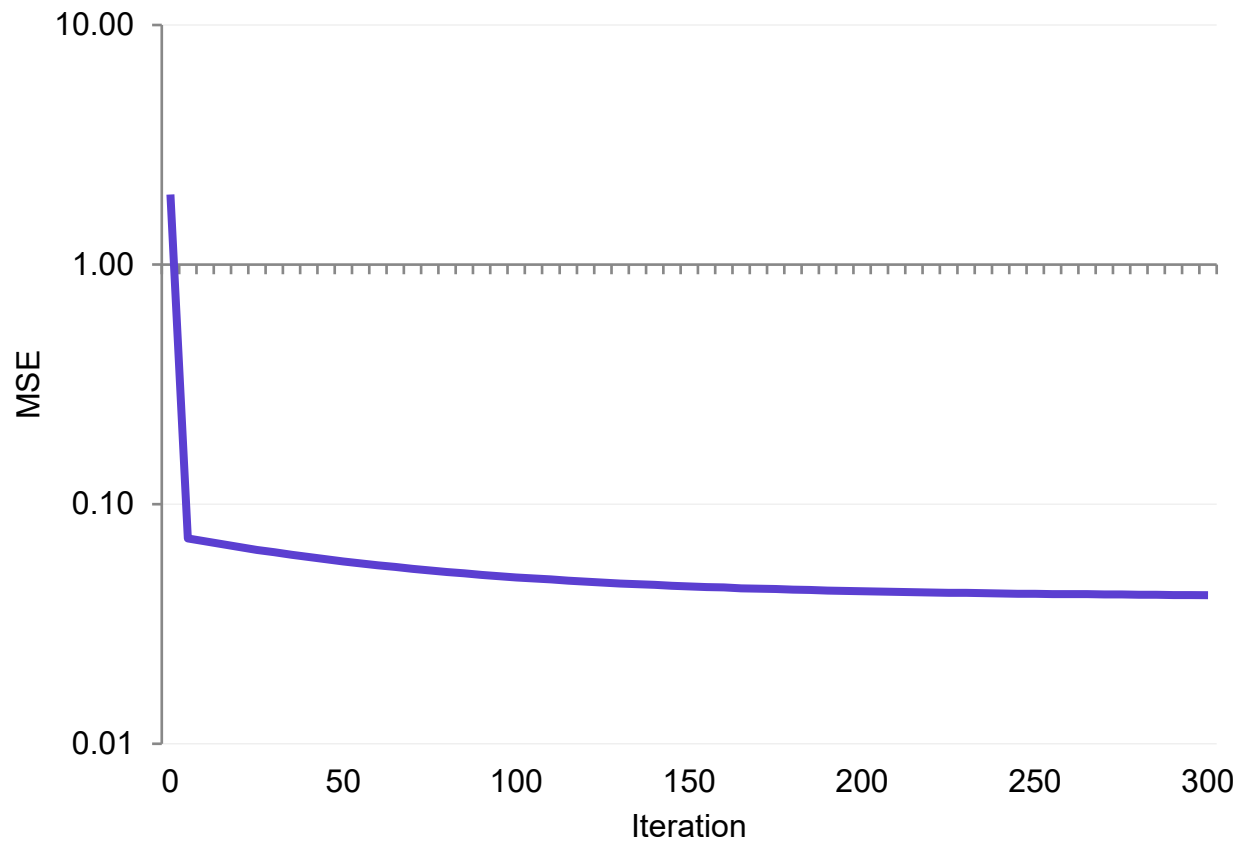
x = X_train[:, 2]       # petal length
y = X_train[:, 3]       # petal width
b0, b1 = 0.0, 0.0       # start at zero
alpha = 0.02            # learning rate
n = len(x)

for epoch in range(3000):
    y_hat = b0 + b1 * x
    error = y_hat - y
    grad_b0 = (2 / n) * error.sum()
    grad_b1 = (2 / n) * (error * x).sum()
    b0 -= alpha * grad_b0
    b1 -= alpha * grad_b1

print(round(b0, 3), round(b1, 3))   # -0.357 0.413
```

Gradient descent in action on Iris

Training MSE per iteration (α = 0.02, log scale)



Parameters converge to the OLS solution

Iteration	β_0	β_1	MSE
0	0.000	0.000	1.9615
1	0.047	0.227	0.2319
10	0.042	0.325	0.0699
100	−0.142	0.365	0.0495
1,000	−0.356	0.413	0.0412
3,000	−0.357	0.413	0.0412

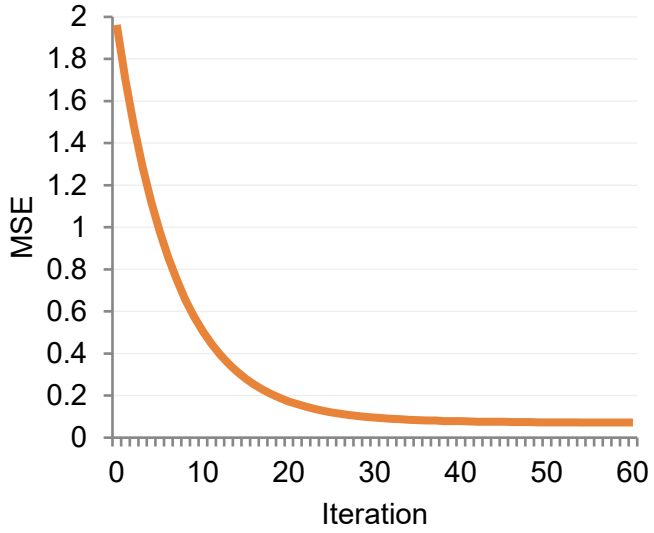
Same answer

After about 1,000 iterations gradient descent reaches $\beta_0 = -0.357$ and $\beta_1 = 0.413$ — identical to the OLS formula.

Choosing the learning rate α

Too small

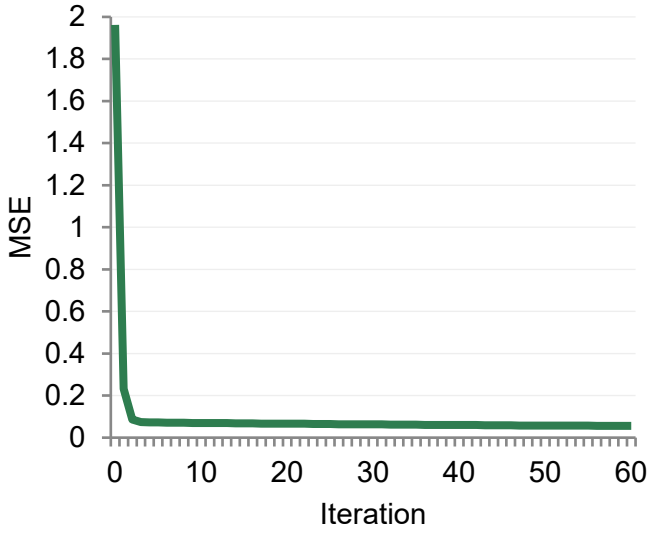
$\alpha = 0.002$



Slow — after 60 steps it is still far from the minimum.

About right

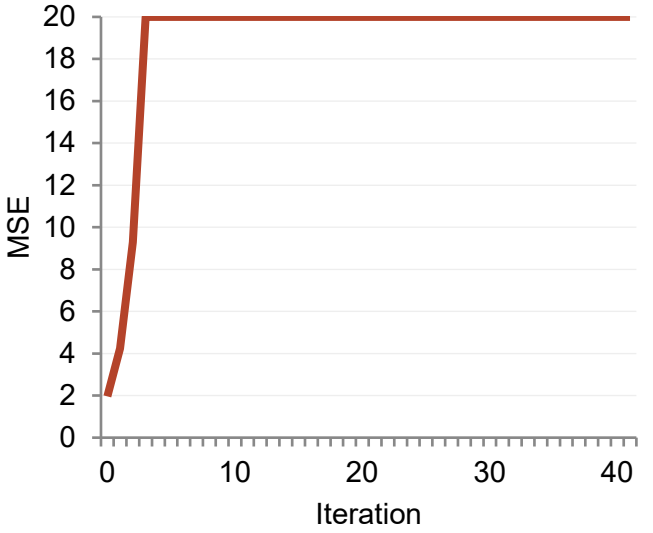
$\alpha = 0.02$



Fast, steady decrease towards the minimum.

Too large

$\alpha = 0.07$



Overshoots the minimum and the loss explodes (chart capped at 20).

Linear regression in scikit-learn

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X, _ = load_iris(return_X_y=True)
x = X[:, [2]]           # petal length, kept 2-D
y = X[:, 3]             # petal width

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(x_train, y_train)

print("slope    :", model.coef_[0])     # 0.4132
print("intercept:", model.intercept_)   # -0.3567
y_pred = model.predict(x_test)
```

X must be 2-D

X[:, [2]] keeps a column shape (n, 1); y can be 1-D.

fit() learns β

scikit-learn solves the OLS formula directly — no learning rate needed.

predict() applies the line

ŷ = intercept_ + coef_ × x for every test flower.

How good is the line? Regression metrics

MAE **0.168 cm**

$\Sigma|y_i - \hat{y}_i| / n$

Average size of an error

MSE **0.046**

$\Sigma(y_i - \hat{y}_i)^2 / n$

Penalises large errors

RMSE **0.214 cm**

$\sqrt{MSE}$

Error in original units

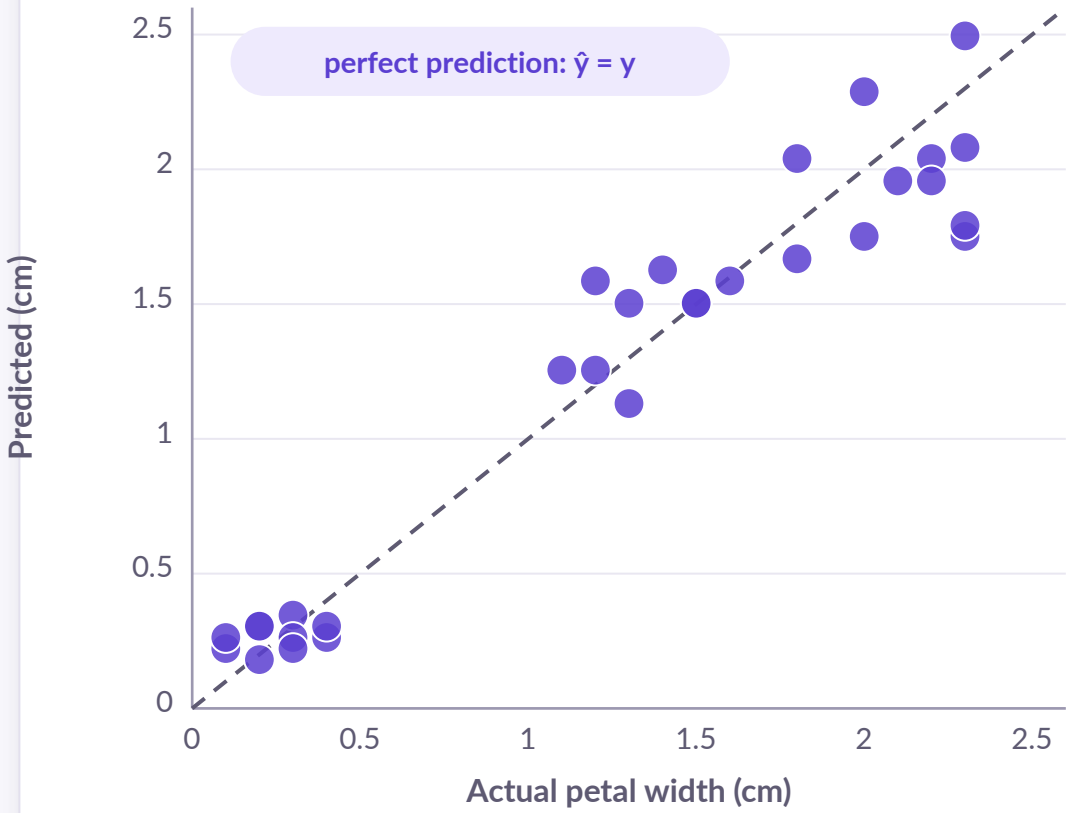
R^2 **0.928**

$1 - SSres / SStot$

Share of variance explained

Reading R^2: 0.928 means the line explains about 93% of the variation in petal width on 30 unseen test flowers. 1 is perfect; 0 is no better than predicting the mean.

Predicted vs actual — 30 test flowers



PART 4

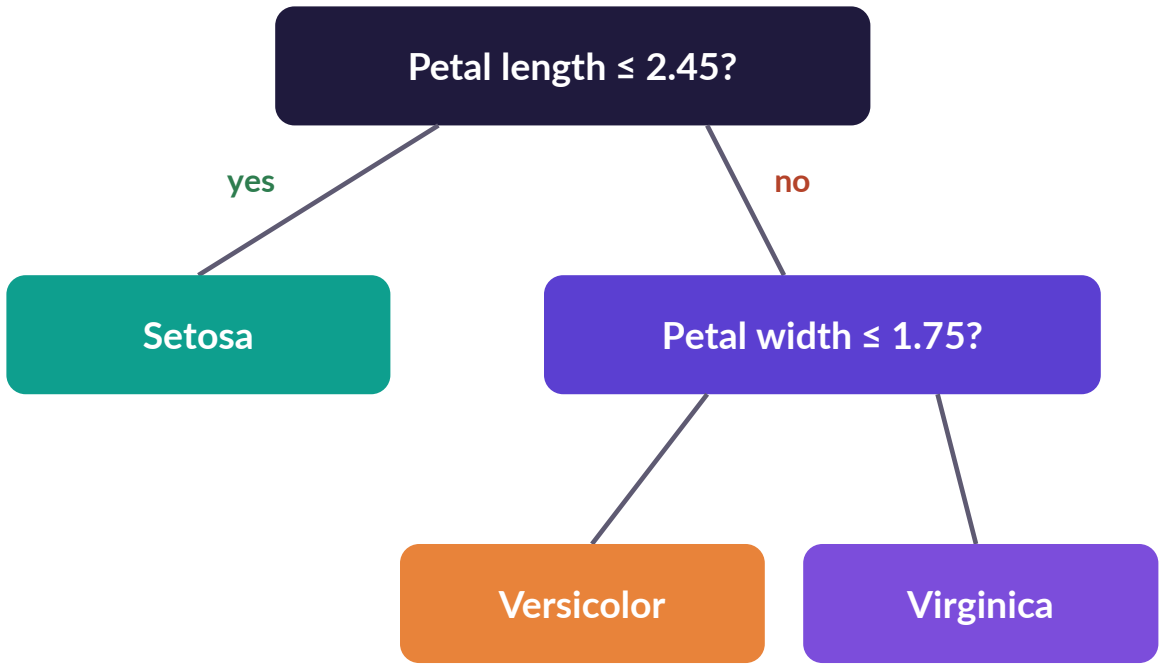
Decision trees



A flowchart of yes/no questions learned from data — and the Gini index that chooses each question.

- Tree anatomy
- Gini index
- Choosing splits
- Growing the tree
- Depth

How a decision tree is generated



Root node first and most informative question

Leaf node final prediction (majority class)

Internal node a further test on a feature

Branch the answer: yes or no

1. Start at the root with all training data
2. Try every feature and every threshold
3. **Pick the split with the lowest weighted Gini**
4. Send data left/right and repeat on each child
5. Stop when a node is pure or a limit is reached

Measuring impurity with the Gini index

GINI IMPURITY OF A NODE

$$Gini = 1 - \Sigma\, p_k^2$$

What it means

- p_k is the proportion of class k in the node
- Gini = 0 → the node is pure (one class)
- Higher Gini → classes are more mixed
- Maximum for 3 equal classes = 1 − 1/3 = 0.667
- Interpretation: chance of mislabelling a random sample

Pure node

$1 - (1^2 + 0^2 + 0^2)$



50 / 0 / 0

0.000

Two classes, 50 / 50

$1 - (0.5^2 + 0.5^2)$



0 / 50 / 50

0.500

Three equal classes (root)

$1 - 3 \times (1/3)^2$



50 / 50 / 50

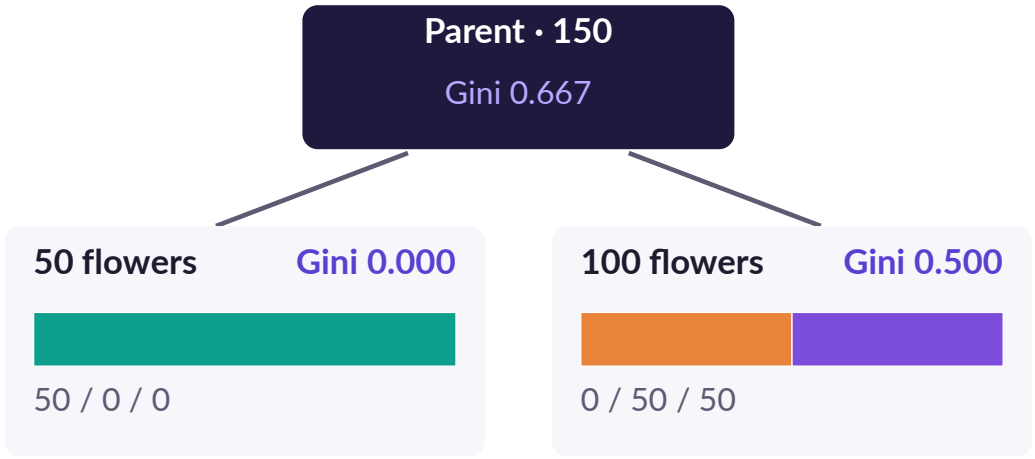
0.667

Choosing the best split with weighted Gini

$$Weighted\ Gini = (nL / n) \cdot GiniL + (nR / n) \cdot GiniR$$
$$Gain = Gini(parent) - Weighted\ Gini$$

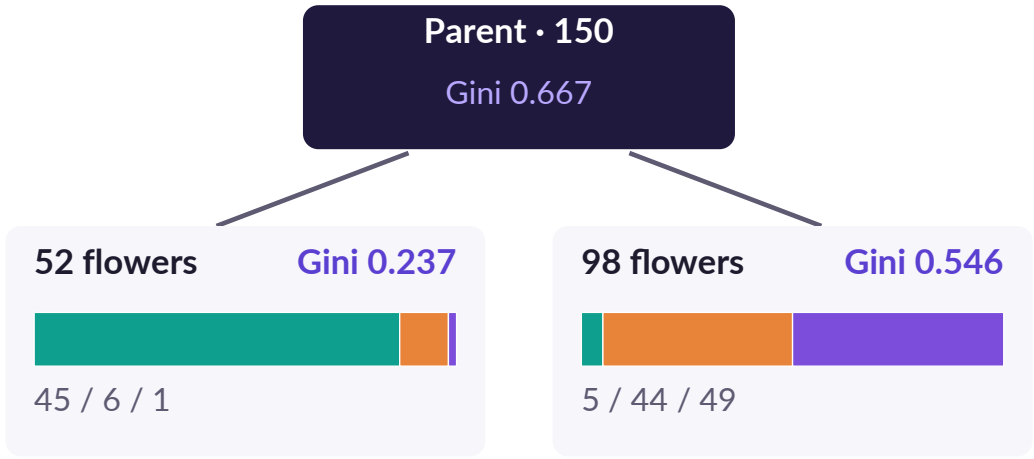
Split A: petal length ≤ 2.45

BEST



Weighted Gini = 0.333 Gain = 0.333

Split B: sepal length ≤ 5.45



Weighted Gini = 0.439 Gain = 0.228

The tree tests every threshold of every feature

Weighted Gini for each petal-length threshold



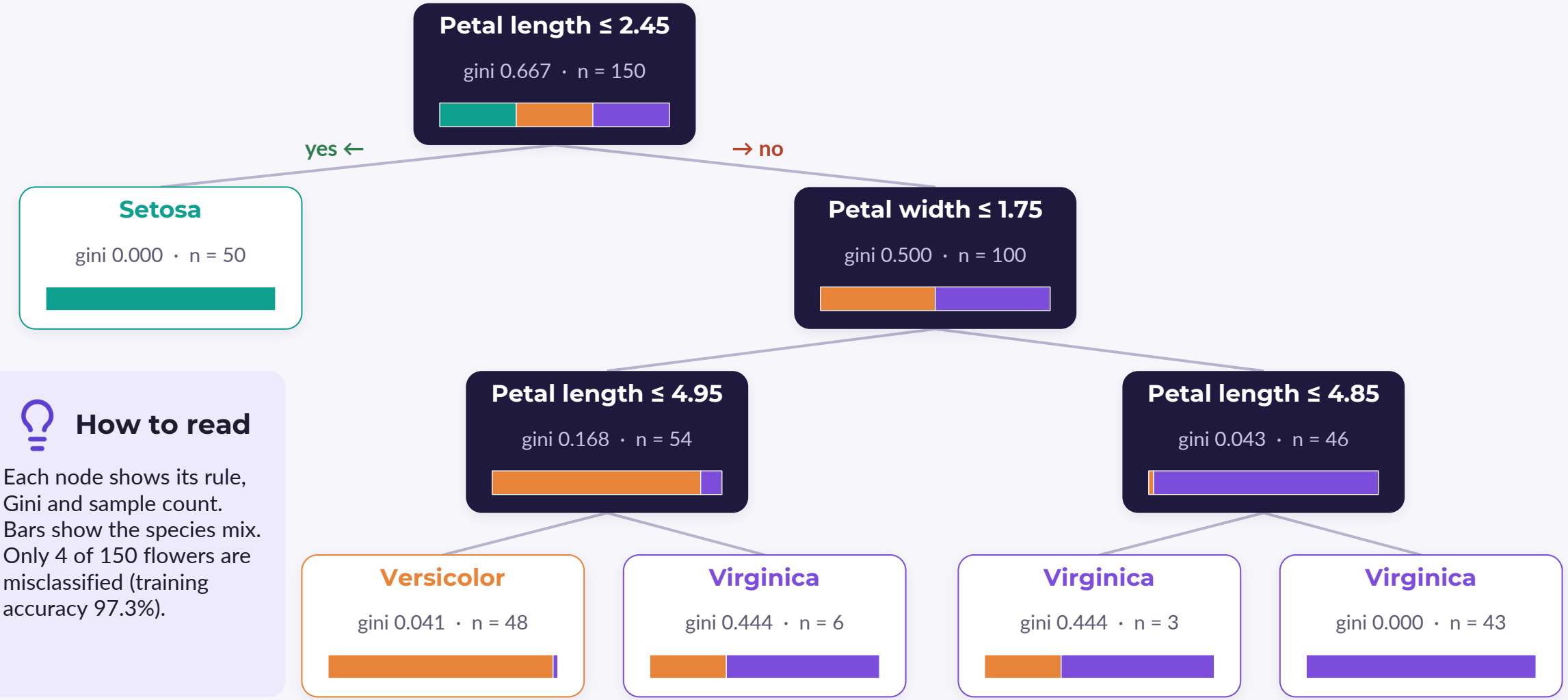
Best split for each feature (all 150 flowers)

Feature	Best threshold	Weighted Gini	Gain
Sepal length	≤ 5.45	0.439	0.228
Sepal width	≤ 3.35	0.540	0.127
Petal length	≤ 2.45	0.333	0.333
Petal width	≤ 0.80	0.333	0.333

A tie

Petal length ≤ 2.45 and petal width ≤ 0.8 both isolate all 50 Setosa flowers perfectly. Either gives a gain of 0.333, so the tree may choose either one.

The decision tree learned from Iris (depth 3)

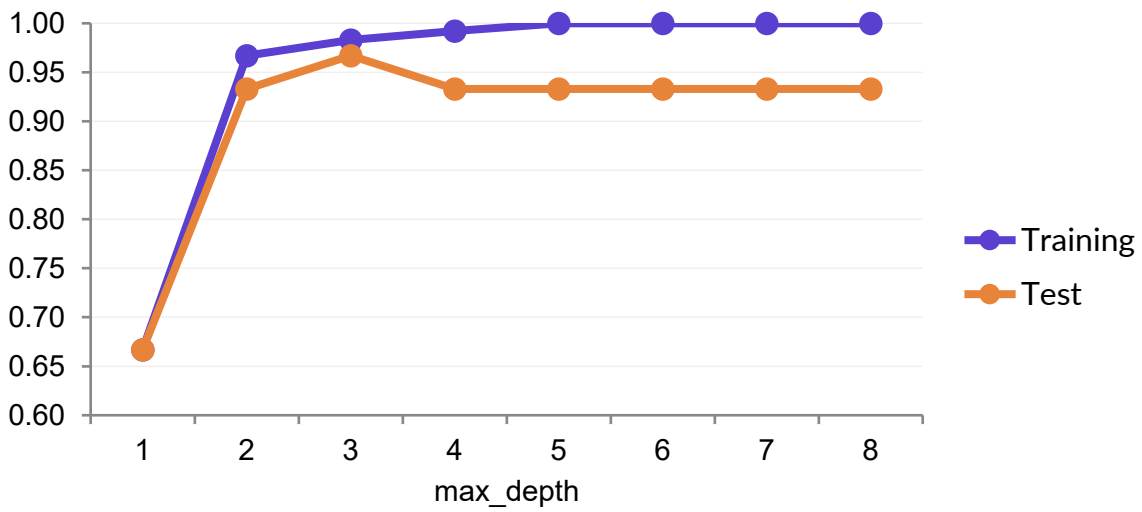


How to read

Each node shows its rule, Gini and sample count. Bars show the species mix. Only 4 of 150 flowers are misclassified (training accuracy 97.3%).

When to stop growing: depth and overfitting

Accuracy vs maximum depth (80 / 20 split)



Hyperparameters that control growth

`max_depth`	maximum number of levels
`min_samples_split`	samples needed to split a node
`min_samples_leaf`	samples required in each leaf
`criterion`	"gini" (default) or "entropy"

Overfitting: deeper trees memorise the training data (100% at depth 5) while test accuracy falls to 0.933. Depth 3 gives the best test score: 0.967.

```python
from sklearn.tree import DecisionTreeClassifier, plot_tree

tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    random_state=42,
)
tree.fit(X_train, y_train)
print(tree.score(X_test, y_test))   # 0.967
plot_tree(tree, feature_names=iris.feature_names)
```

PART 5

Unsupervised & reinforcement learning

Learning without labels — and learning by trial, error and reward.



Unsupervised

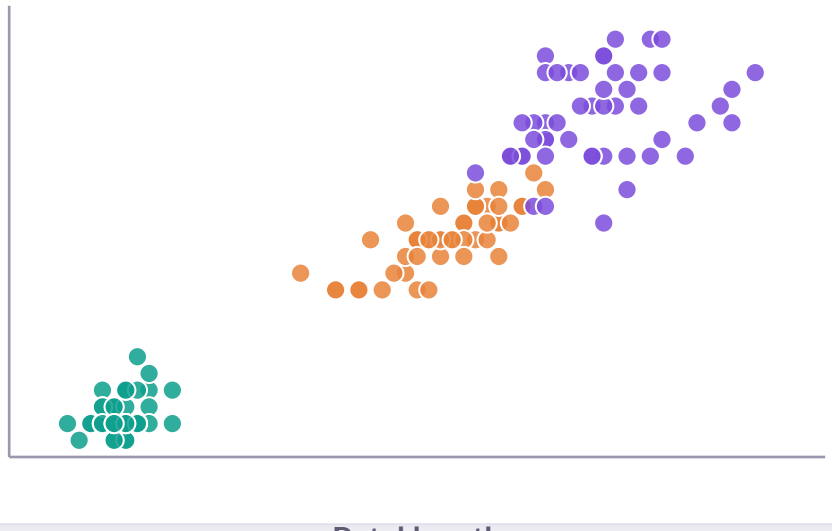
K-means

Elbow method

Reinforcement

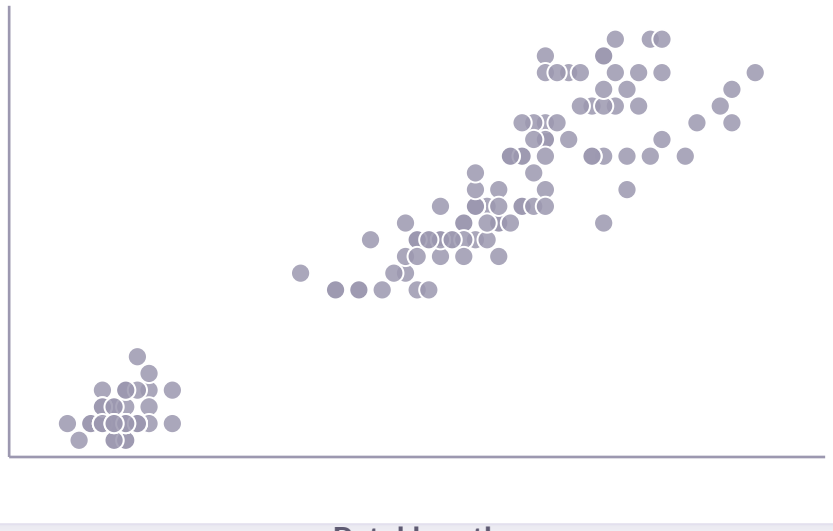
What is unsupervised learning?

Supervised: we know each species



remove y

Unsupervised: only X, no labels



The algorithm must find structure in the data by itself — there is no “correct answer” to learn from.

Clustering

Group similar observations: k-means, hierarchical, DBSCAN

Dimension reduction

Compress many features into a few: PCA, t-SNE

Anomaly detection

Find unusual points: fraud, faulty sensors

The k-means algorithm in four steps

1 — Choose K

Decide how many clusters to find (for example K = 3).

2 — Initialise

Place K centroids — often at randomly chosen data points.

3 — Assign

Give each point to its nearest centroid (Euclidean distance).

4 — Update

Move each centroid to the mean of the points assigned to it.

repeat 3 → 4 until centroids stop moving

DISTANCE TO A CENTROID

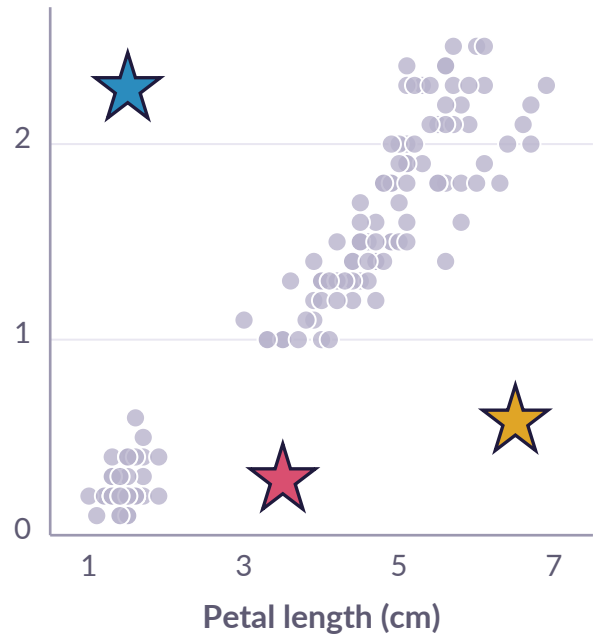
$$d(x, \mu) = \sqrt{\Sigma_j \, (x_j - \mu_j)^2}$$

OBJECTIVE: INERTIA, X IN CLUSTER K (MINIMISE)

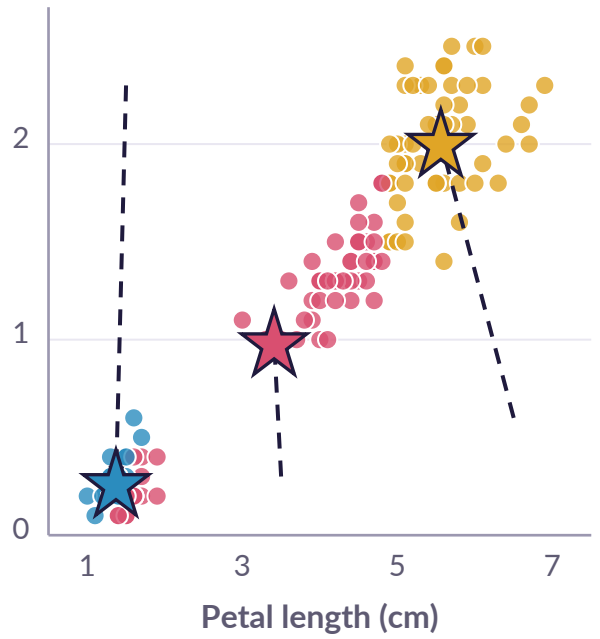
$$WCSS = \Sigma_k \, \Sigma_x \, \|x - \mu_k\|^2$$

Watching k-means converge on petal measurements

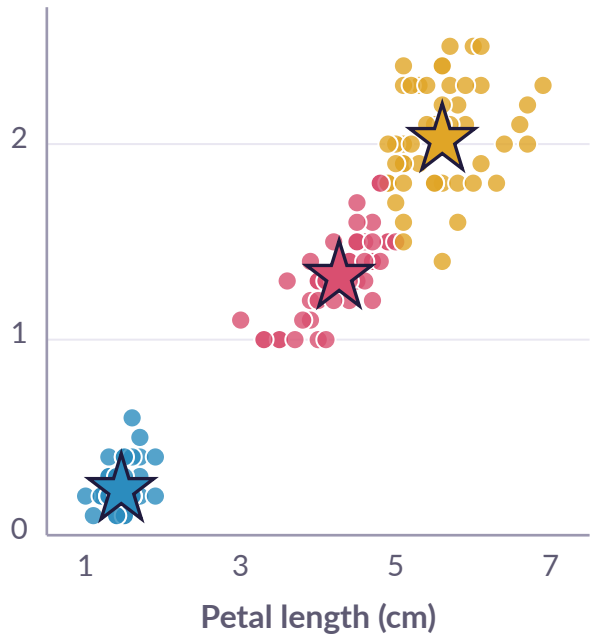
1 Start: random centroids



2 After iteration 1



3 Converged (8 iterations)



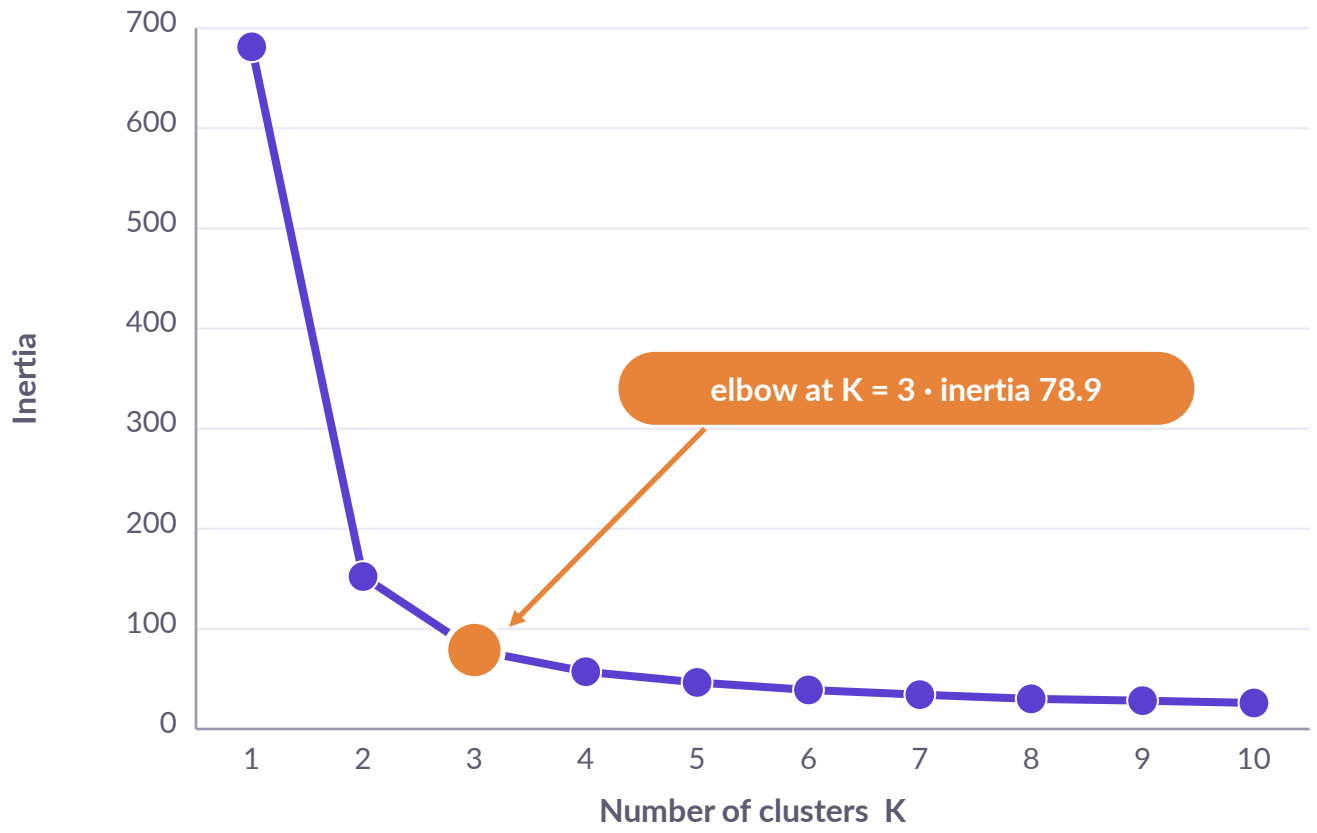
Stars are centroids. They begin in poor positions on purpose.

Centroids move. Each moves to the mean of its points.

Stable result. Assignments stop changing: converged.

The elbow method: how many clusters?

Inertia (WCSS) for K = 1 … 10 — all four features



Inertia

Sum of squared distances from each point to its centroid. Lower = tighter clusters.

It always falls

More clusters always reduce inertia: 681.4 → 152.3 → 78.9 → 57.2.

Find the bend

Choose K where adding another cluster gives little improvement — here K = 3, matching the 3 species.

python

```
inertia = []
for k in range(1, 11):
    km = KMeans(k, n_init=10, random_state=42)
    inertia.append(km.fit(X).inertia_)
```

Did k-means rediscover the species?

Clusters found (K = 3) compared with the true species

	Setosa	Versicolor	Virginica
Cluster 1	50	0	0
Cluster 2	0	48	14
Cluster 3	0	2	36

```python
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score

km = KMeans(n_clusters=3, n_init=10, random_state=42)
clusters = km.fit_predict(X)        # y is never used

print(km.cluster_centers_)
print(adjusted_rand_score(y, clusters))   # 0.73
```

134 / 150

flowers in the matching cluster

0.73

adjusted Rand index (1 = perfect)

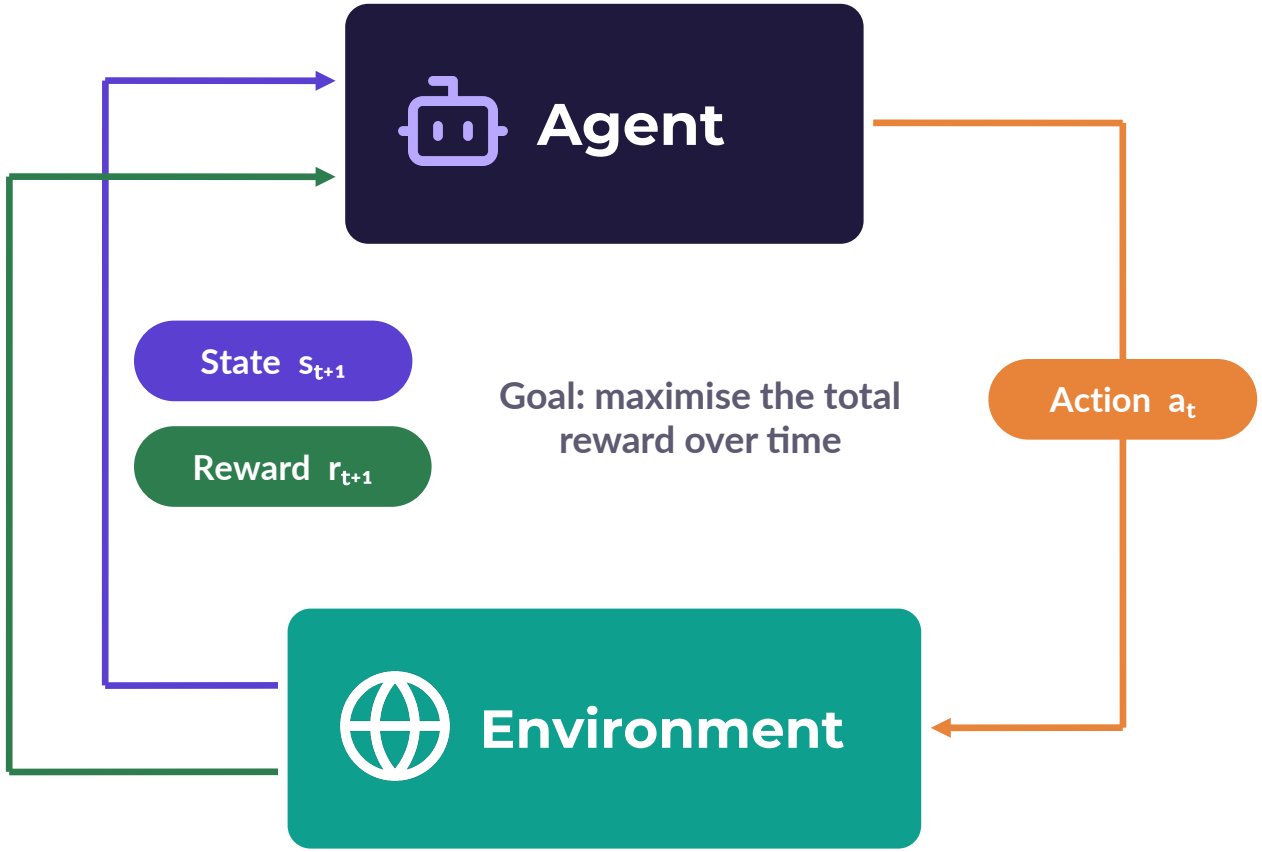
0

labels used for training

Interpretation

Setosa is recovered perfectly. Versicolor and Virginica overlap, so 16 flowers land in the “wrong” cluster — k-means only sees distances, not species.

Reinforcement learning: learning from rewards



No dataset of correct answers. The agent tries actions, observes the result and gradually learns a policy — a strategy for which action to take in each state.

Games

AlphaGo learned to beat world champions at Go.

Robotics & driving

Learn to move, grasp objects or stay in lane.

Operations

Traffic lights, data-centre cooling, trading.

Key ideas in reinforcement learning

Agent

The learner and decision maker

Environment

Everything the agent interacts with

State s

The current situation

Action a

A choice the agent can make

Reward r

Numerical feedback after an action

Policy π

Mapping from states to actions

Q-LEARNING UPDATE

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \cdot max_{a'} Q(s', a') - Q(s, a) \right]$$

α	learning rate
γ	discount for future rewards
$Q(s, a)$	expected total reward of action a in state s

Exploration vs exploitation

Exploit: take the action that is known to be good. Explore: try something new that might be better. An ε-greedy agent explores at random with probability ε and exploits otherwise.

PART 6

Evaluating classifiers

Accuracy alone can mislead. The confusion matrix shows exactly where a model goes wrong.

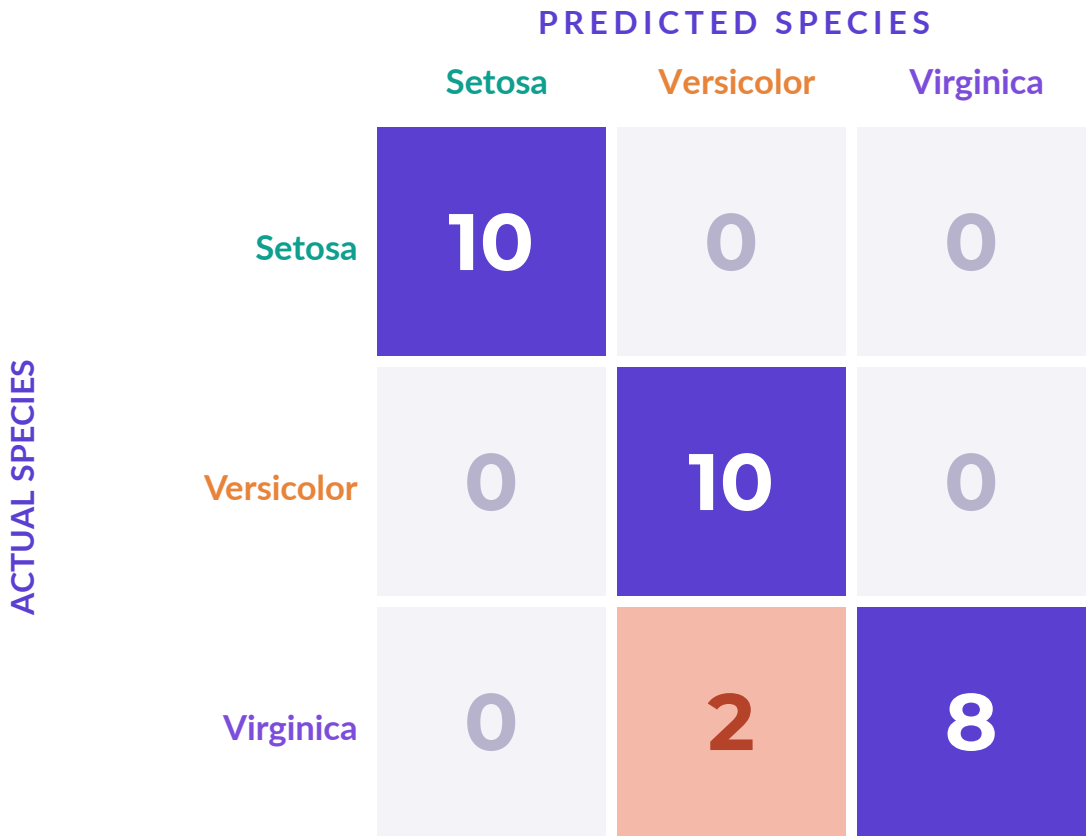


- Confusion matrix
- Accuracy
- Precision
- Recall
- F1-score

PERFORMANCE METRICS

The confusion matrix

KNN (k = 5, scaled) on 30 unseen test flowers



Rows = truth · columns = prediction

28 correct

The diagonal holds every correct prediction.

2 errors

Two Virginica flowers were predicted as Versicolor.

Setosa: perfect

All 10 Setosa flowers were classified correctly.

PERFORMANCE METRICS

TP, FP, FN and TN: one class at a time

Treat Versicolor as the “positive” class and the other two species as “negative”.

	Predicted Versicolor	Predicted other
Actually Versicolor	TP **10** True positive	FN **0** False negative
Actually other	FP **2** False positive	TN **18** True negative

TP — True positive

Versicolor correctly predicted as Versicolor.

FN — False negative

A Versicolor flower the model missed.

FP — False positive

Another species wrongly called Versicolor — the 2 Virginica.

TN — True negative

Other species correctly predicted as not Versicolor.

PERFORMANCE METRICS

Accuracy, precision, recall and F1-score

Accuracy

How often is the model right overall?

0.933

$$(TP + TN) / total = 28 / 30$$

Precision

When it says Versicolor, how often is it right?

0.833

$$TP / (TP + FP) = 10 / 12$$

Recall

Of all real Versicolor, how many were found?

1.000

$$TP / (TP + FN) = 10 / 10$$

F1-score

Balance of precision and recall (harmonic mean)

0.909

$$2 \cdot P \cdot R / (P + R) = 2 \times 0.833 \times 1.0 / 1.833$$

PERFORMANCE METRICS

The classification report — and which metric to trust

Class	Precision	Recall	F1-score	Support
Setosa	1.000	1.000	1.000	10
Versicolor	0.833	1.000	0.909	10
Virginica	1.000	0.800	0.889	10
Macro average	**0.944**	**0.933**	**0.933**	30
Accuracy			0.933	30

```python
from sklearn.metrics import (confusion_matrix,
    classification_report, f1_score)

y_pred = model.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred,
      target_names=iris.target_names, digits=3))
```

Which metric when?



Accuracy

Classes are balanced and all errors cost the same — like Iris.



Precision

False positives are costly — e.g. a spam filter hiding a real email.



Recall

False negatives are costly — e.g. missing a disease in screening.



F1-score

Classes are imbalanced and you need both precision and recall.

KEY TAKEAWAYS

What you should now be able to explain

Machine learning

Algorithms learn rules from data (E) to improve at a task (T), measured by P.

Features & variables

Features X are independent inputs; the target y is the dependent output.

The pipeline

Define → collect → explore → clean → engineer → split & scale → train → evaluate.

Data preparation

Split before scaling; fit the scaler on training data only to avoid leakage.

Linear regression

$\hat{y} = \beta_0 + \beta_1 x$, fitted by minimising MSE with OLS or gradient descent.

Decision trees

Recursive splits chosen by the lowest weighted Gini; limit depth to avoid overfitting.

K-means

Assign points to the nearest centroid, update the means; pick K with the elbow.

Evaluation

Read the confusion matrix; choose accuracy, precision, recall or F1 to suit the costs.

KEY TAKEAWAYS

Acknowledgement

This lecture was inspired by the following material:
- Machine Learning (Applied AI 25/26 – Dr. Ester Bonmati)